

I. Cloud Computing Fundamentals and Stack

A. Definition and Concept

- Cloud computing is often referred to as a “**stack**” because it encompasses different types of services built upon one another.
- The National Institute of Standards and Technology (NIST) defines it as a model that enables **on-demand access to a shared resource pool** (servers, networks, applications, services, storage) which can be rapidly deployed with minimal management.
- Nearly every online application uses cloud services, and companies are shifting business operations to the cloud using a services-oriented methodology.
- Users can utilize bulk resources that can be obtained quickly whenever demand arises.

B. NIST Essential Features

For services to be considered cloud hosting services, NIST identifies these essential features:

- **On-demand services:** End users can sign up for and obtain services without delays.
- **Wide network access:** Users can access services through multiple platforms (laptops, desktops, mobiles).
- **Measured services:** Users pay based on usage; billing is offered as a utility service.
- **Elasticity and scalability:** The system can cope with increases in demand.

C. Cloud Computing Service Models (Stack Layers)

There are three main types of cloud computing services:

1. **Software as a Service (SaaS):** Applications designed for end users, offered across the Internet.
2. **Platform as a Service (PaaS):** Sets of tools or services designed for efficiently coding and implementing applications.
3. **Infrastructure as a Service (IaaS):** Software and hardware that powers the cloud, such as servers, networks, storage, and operating systems.

II. Detailed Exploration of Cloud Service Models (IaaS, PaaS, SaaS)

A. Software as a Service (SaaS)

- **Definition:** Software deployed over the web, licensed by a vendor to clients as an on-demand service.
- **Model:** Offered through a subscription (pay-as-you-use) model or sometimes free of cost (generating revenue via advertisements).
- **Management:** Managed from a central point, uses a "one-to-many" model, and users do not worry about patches or upgrades.
- **Suitable Applications:** "Vanilla" offerings (e.g., email), applications demanding Internet access (e.g., sales management), applications involving outside world interplay (e.g., email newsletter campaigns), software with frequent demand spikes (e.g., tax software), or software needed for a short term (e.g., collaboration tools).
- **Not Suitable Applications:** Applications needing fast processing of real-time data, applications where laws prohibit external data hosting, or applications with sufficient on-site solutions.
- **Examples:** Salesforce, Google Workspace, Dropbox, Microsoft Office 365.

B. Platform as a Service (PaaS)

- **Definition:** A computing model allowing fast creation of applications without buying or maintaining software and infrastructure. PaaS is the platform used for creation of software, unlike SaaS which is the software delivered.
- **Key Features/Services:**
 - Integrated development setting for developing, testing, implementing, and maintaining applications.
 - Tools for web-based interface creation, testing, and deployment.
 - Tools for handling billing.
 - Built-in scalability, including failover and load balancing.
 - Integration with databases and web services through common standards.
 - Supports multitenant architecture and development team collaborations (some offer project planning tools).
- **Suitable Use:** Mainly used when many developers work on one project, when outside parties need to communicate with development processes, or when leveraging existing data sources to build applications.
- **Not Suitable Use:** When the application must be portable, or when proprietary languages may affect development.
- **Examples:** Heroku, Google App Engine, Microsoft Azure App Service.

C. Infrastructure as a Service (IaaS)

- **Definition:** Cloud computing infrastructure (servers, storage, operating systems, network) purchased as an on-demand service instead of buying the physical infrastructure.
- **Deployment Models:** Includes public cloud (shared resources deployed across the Internet), private cloud (resources offered through a private network), and hybrid cloud (combination of both).

- **Cost and Scaling:** Resources are distributed as services, allowing dynamic scaling. It has **variable costs** following a utility pricing model. IaaS is generally a bit **costlier** than PaaS or SaaS.
- **Suitable Use:** When demands are changing, for new businesses lacking capital for hardware investment, when an organization is growing fast and scaling hardware is challenging, when needing to reduce capital costs and shift to operational costs, or for specific/short-term infrastructure needs.
- **Not Suitable Use:** Where data storage outsourcing is difficult due to regulatory compliance, or where on-site infrastructure meets organizational needs.
- **Examples:** Amazon Web Services (AWS), Microsoft Azure, Google Cloud.
- **Control:** IaaS gives users control over both applications and the infrastructure. The vendor spends on physical servers, networking, and storage.

D. Comparison of Cloud Service Models (IaaS vs PaaS vs SaaS)

The models are differentiated mainly by **control** and **cost**.

Attribute	IaaS	PaaS	SaaS
Overview	Provides virtualized computing resources.	Offers platform for developers to build, deploy, manage apps without dealing with infrastructure.	Delivers software applications over the internet without installation/maintenance.
Responsibility (Managed by User)	Users manage applications, data, runtime, middleware, and OS.	Users focus on application development and data.	Users only configure and use the software.
Responsibility (Managed by Provider)	Provider manages infrastructure.	Provider manages runtime, middleware, OS, and infrastructure.	Provider manages everything (application, runtime, data, middleware, OS, infrastructure).
Control/Suitability	High flexibility; users control entire stack.	More control over applications/data than SaaS vendor. Better for app development enterprises.	Least flexibility; minimal customization. Most suitable for small enterprises.

Scalability	Highly scalable; resources added/removed as needed.	Offers automatic scalability.	Scalability is entirely managed by the service provider.
Development Time	Longer, as users manage infrastructure components.	Shorter, as infrastructure management is abstracted.	Shortest, as users only configure and use the software.

III. Cloud Deployments for Large Businesses

A. Hybrid Cloud Model

- A **hybrid cloud deployment model** is often considered the best option for large-scale businesses.
 - **Benefits:** It combines the cost savings and scalability of the public cloud with the control and security of a private cloud.
 - **Usage:** Public cloud is used for non-sensitive data and applications (e.g., email, file sharing), while sensitive data and applications are kept in the more secure private cloud to meet compliance requirements.
 - **Agility:** Hybrid cloud allows organizations to leverage public cloud services (e.g., data analytics, machine learning, IoT) while still using their own resources, promoting agility and responsiveness.
 - **Financial Advantage:** Provides greater scalability and flexibility, allowing businesses to pay only for the resources they need, reducing the need to invest in expensive, potentially underutilized infrastructure.
-

IV. Connecting to Cloud Servers

- Cloud Service Providers (CSPs) own and operate data centers worldwide, housing the physical infrastructure. Access to this infrastructure is typically on a pay-as-you-go basis.
- Users connect to a Cloud Server using methods determined during deployment, such as password authentication or an SSH key.

A. OpenSSH (for Unix-based systems)

- **Primary Method:** OpenSSH is the primary way to connect to a Linux server using an SSH client.

- **Usage:** Available directly through the local terminal for Unix-based systems (macOS, Linux, WSL).
- **Command:** The connection command consists of three parts: `ssh`, username, and remote host (e.g., `ssh root@remote-host`).
- **Authentication:**
 - If an SSH key is configured, the system attempts to authenticate using the key. If successful, the user is logged in straightaway.
 - If no SSH key is configured, the system reverts to **password authentication** and prompts for the server's one-time password (OTP). Users must change the OTP immediately upon successful login. Password characters are not displayed on the screen for security.

B. PuTTY (for Windows)

- PuTTY is a commonly used SSH client for Windows users who lack a built-in solution for SSH.
- **Steps:** Enter the server's public IP address in the Hostname field. Navigate to `Connection > Data` and enter the auto-login username (usually `root` for new Linux servers).
- **SSH Key:** Navigate to `Connection > SSH > Auth > Credentials` to select the private key used during deployment.
- **Saving Configuration:** Configurations can be saved on the Session screen for future use by giving it a name and clicking Save.
- **Password Authentication:** If no SSH key was configured, enter the public IP address and click Open to be prompted for the password.

C. Other Connection Methods

- **Remote Desktop Connection:** Used for Windows servers, allowing operation like a desktop. Uses the username `Administrator` and the password generated at deployment.
- **Console Connection:** An HTML5-based web console available at the Cloud Control Panel. Useful when SSH access fails due to faulty firewall rules or OS errors.
- **VNC Console:** An optional access method using a VNC client. Requires enabling the VNC service and setting a password/keyboard mapping in the Control Panel. Changing the keyboard map requires the server to be powered down.

V. Service-Oriented Architecture (SOA)

A. Definition and Components

- **SOA** is a methodology of software development that uses software components called **services** to create business applications.
- SOA provides a specification and methodology for offering platform and language-independent services for use in distributed applications.
- A **service** is a repeatable task within a business process. A business task is a composition of services.
- **Client Interaction:** Clients access SOA-compliant components by passing a message (containing metadata, often an XML file transported via SOAP). The component acts on the message and returns a response.
- **Endpoint:** The component exposes itself as an “**endpoint**” to the client.
- **Abstraction:** The client views the service component as a **black box**; the nature of the component’s processing, location, operating systems, and applications involved are unknown. This abstraction allows components to be upgraded, replaced, or moved without disrupting clients.
- **Core Values (Six Key Parameters):** Business value, strategic goals, intrinsic inter-operability, shared services, flexibility, and evolutionary refinement.
- **Gain/Loss:** SOA provides the ability to add significant capabilities with fraction of the cost/effort. It involves losing the ability to perform fundamental customization of the service when provided by a third party.

B. Messaging Protocols and Standards

- Message passing requires a **data interchange format** and a **network protocol**.
- **WSDL (Web Services Description Language):** Commonly used to describe the service interface, binding information, and the nature of the component’s service/endpoint. Version 1.1 is a W3C standard. WSDL files usually contain XML data and an XML schema.
 - **Essential Objects in WSDL:** Service object, port/endpoint (unique address), binding (interface/transport description), portType (defines capabilities/operations), operation, message content, and types.
- **SOAP (Simple Object Access Protocol):** The most commonly used message-passing format using Extensible Markup Language (XML). It exposes Web objects through an API and provides a more precise and error-free consumption model, often accompanied by sophisticated development tools.
- **REST (Representational State Transfer):** Offers light weight access using standard HTTP commands (GET, POST, PUT, DELETE) and is easier to implement with less overhead than SOAP. All major Web services use REST, and many combine it with SOAP.
- **Service Contract:** Codifies the relationship between the data, metadata, intended service, and how the service acts on the message.
 - **Contract Components:** Header (name, version, owner, responsibility, e.g., RACI matrix); Service Type (data, business, integration, presentation, process); Functional Specification (requirements, URL, invocation method); Transaction Attributes (QoS, SLA, security parameters).

C. The Enterprise Service Bus (ESB)

- **Function:** Middleware software often employed in SOA implementations to act as a **transaction manager** (broker) and translator.
- **Architecture:** An ESB is an architectural pattern (not a physical network bus) comprised of network services that manage transactions. It separates clients from components and ensures messages get where they need to go.
- **Mediation Services:** ESB performs message translation, registration, routing, logging, auditing, and managing transactional integrity (similar to ACID in databases).
- **Environment:** Creates a virtual environment layered on top of an enterprise messaging system where services are advertised and accessed.
- **Typical Features:** Monitoring, process management, data repositories/registries, data services, data abstraction (translation), governance (compliance monitoring), and security services.
- **Repository vs. Registry:**
 - **Repository:** Stores references to SOA components, source code, and linking information. Acts like a name server.
 - **Registry:** Contains references to rules, descriptions, and definitions of services (metadata). Acts like a directory service (domain).

D. Combining Web Services: Orchestration vs. Choreography

These are the two main methods for combining Web services:

- **Orchestration:** A middleware service (orchestrator) centrally coordinates all different Web service operations. All services send and receive messages from the orchestrator.
 - **Advantages:** Centralized management of business logic, easier integration of services (as they don't need to know the business process), and simpler error handling.
 - **Implementation:** Often performed through an **Enterprise Service Bus (ESB)**.
- **Choreography:** Has no central coordination function. Each Web service is aware of when to process a message and which component to interact with.
 - **Logic:** A collaborative effort where the business process logic is pushed out to the members.
- **Preference:** Most mature SOA implementations favor orchestration over choreography.

E. Event-Driven SOA (SOA 2.0)

- **Definition:** An extension of SOA designed to respond to events that occur as a result of (or influence) a business process.
- **Functionality:** Can allow low-level events to trigger a business process, correlate events with SOA design information, inhibit a process if events don't appear, or invoke a reaction based on a trigger.

- **Causal Vector Engine (CVE):** To perform these tasks, an engine with built-in artificial intelligence must be added to the SOA design. The CVE analyzes event sequences, relationships, and timing using a set of built-in rules.
-

VI. Business Process Languages and Modeling

A. Business Process Execution Language (BPEL/WS-BPEL)

- **Purpose:** The standard language for Web service interactions, used for grouping and managing sets of atomic transactions to execute a business process.
- **Functions:** A meta-language comprised of executable commands for Web services/clients and internal/abstract code for executing internal business logic.
- **Usage:** Used to compose, orchestrate, and coordinate business processes with Web services in SOA, supporting asynchronous communications.
- **Technology Base:** Uses XML and supports messaging protocols like SOAP, WSDL, and UDDI. BPEL built on IBM's WSFL and Microsoft's XLANG.
- **Structure:** Has an internal (executable) and external (abstract) view. Data functions support process data, control flow, logic/branching structures, and process orchestration.

B. Business Process Modeling Methodologies

Various modeling tools and methodologies have been developed to support SOA development and optimization.

Modeling Tool	Description
Unified Modeling Language (UML)	Standard by OMG. Creates graphical representations of software systems using diagram types (structural and behavior types). Widely used; developed models can be automatically reduced to code.
XML Metadata Interchange (XMI)	OMG standard used to exchange metadata using XML. UML models often use XMI as their interchange format.
Systems Modeling Language (SysML)	Open-source extension of UML profiles. Smaller and easier to work with than UML.
Business Process Modeling Notation (BPMN)	Methodology for representing business processes as a set of connected visual objects illustrating workflow in a Business Process Diagram (BPD). A BPD can be reduced to the executable WS-BPEL standard.

Service-Oriented Modeling Framework (SOMF)	Combines a modeling language with a graphical display of SOA components, viewing the system as a map of objects and relationships. Allows developers to create action plans and run "what-if" scenarios.
---	--

C. SOMF Concepts

- **Service Life Cycle:** Conceptualization, Discovery, Analysis, Integration, Logical design, Architecture design, and System implementation.
 - **Service Granularity:**
 - **Atomic Service:** Granular, narrow in scope, and cannot be decomposed into smaller useful functions (e.g., a customer lookup).
 - **Composite Service:** A collection of services that work together (e.g., an ESB). Usually organized as a hierarchical topology and is coarse-grained.
 - **Service Cluster:** A collection of composite services that form a process module.
 - **Modeling Topologies:**
 - **Circular Topology:** Message passing is circular; no orchestrator; choreography is maintained at the component level.
 - **Hierarchical Topology:** Services arranged in a tree pattern (parent/child). Advantages: well-defined relationships, central root logic, clear address space. Disadvantage: message passing overhead is not optimized.
 - **Network Topology:** Many-to-many relationship between services and clients. Advantage: overhead minimized. Disadvantage: considerable overhead maintaining links.
 - **Star Topology:** Services connect through a central service. Favored in orchestration processes and useful for broadcasting/multicasting services.
-

VII. SOA Management and Security

A. Managing and Monitoring SOA

- SOA management requires proactive oversight as components are distributed and highly virtualized.
- **Framework Products:** Include HP OpenView SOA Manager, IBM Tivoli Framework Composite Application Manager for SOA (ITCAM), and Oracle BPEL Process Manager.
- **Specialized Tools:** BMC Software's AppSight (automated problem resolution) and Tidal Software's Intersperse (root cause analysis). CA Wily SOA Solution (monitoring and discovery) and iTKO's LISA (testing platform).

B. SOA Security

- SOA eliminates application boundaries, making traditional application-level security less effective.
 - **OASIS Standards:**
 - **SAML (Security Assertion Markup Language):** XML standard providing data authentication and authorization between client and service; used in Single Sign-on Systems (SSO).
 - **WS-Security (WSS):** Extension of SOA that enforces security by applying tokens (Kerberos, SAML, X.509) to messages using XML Signature and XML Encryption.
 - **WS-Trust:** Extends WSS to provide a mechanism to issue, renew, and validate security tokens, often implemented via a Security Token Service (STS).
 - **XML Gateway:** Another security approach that intercepts XML messages (transported by SOAP or REST), identifies the source, and verifies secure reception, typically requiring a Public Key Infrastructure (PKI).
-

VIII. Open Cloud Consortium (OCC) and SOA/Cloud Relation

A. Open Cloud Consortium (OCC)

- **Mission:** An organization supporting the development of standards for cloud computing and interoperating with various frameworks.
- **Functions of Working Groups:**
 - Develop benchmarks for measuring cloud computing performance (e.g., MaStone).
 - Provide testbeds for vendors (e.g., Open Cloud Testbed and Intercloud Testbed).
 - Support the development of open-source reference implementations.
 - Support management of cloud infrastructure for scientific research (e.g., Open Science Data Cloud).
 - Work on standards for Large Data Clouds, often following the Google model using distributed file systems, table services, and MapReduce.

B. Relation between SOA and Cloud Computing

- Although Web services can implement SOA, it is not a requirement for all cloud applications.
- As cloud applications become more diverse, **SOA offers an architectural blueprint** for accessing diverse optimized services through a loosely coupled, standardized method.
- SOA is **loosely coupled** because the service is separated from the messaging. If a component needs to be replaced, switching to a different one is easy and requires almost no programming effort.

- This convergence of SOA and Web 2.0 is expected to create an **"Internet of Services"** where complex services are available as building blocks (Advanced SOA).